

Modélisation Monte Carlo en C++

Sommaire

- Cadre mathématique de la méthode Monte Carlo
- Simulation du sous-jacent sous différents modèles
- Pricing d'un call européen pour chaque modèle
- Pricing de produits exotiques
- Méthodes de réduction de variance
- Parallélisation avec OpenMP

Cadre de valorisation

On se place dans un marché financier vérifiant :

- **Absence d'arbitrage** : il n'existe pas de stratégie autofinancée permettant un gain certain sans risque.
- **Marché complet** : tout payoff atteignable peut être répliqué par une stratégie de portefeuille.
- **Conséquence** : il existe une unique mesure risque-neutre $\mathbb{Q}$.

Le prix d'un produit dérivé est donné par l'espérance actualisée de son payoff sous $\mathbb{Q}$.

Valorisation risque-neutre

Soit $(V_t)_{t \geq 0}$ le prix d'un produit dérivé écrit sur un sous-jacent $(S_t)_{t \geq 0}$.

Son payoff peut dépendre de toute la trajectoire du sous-jacent :

$$\Phi((S_u)_{t \leq u \leq T}).$$

Sous l'unique mesure risque-neutre $\mathbb{Q}$:

$$V_t = \mathbb{E}^{\mathbb{Q}} \left[\frac{B_t}{B_T} \Phi((S_u)_{t \leq u \leq T}) \mid \mathcal{F}_t \right].$$

Actualisation du payoff

L'actif sans risque $(B_t)_{t \geq 0}$ est défini par :

$$B_t = \exp \left(\int_0^t r_s ds \right).$$

Si le taux d'intérêt r est constant :

$$B_t = e^{rt} \quad \text{et} \quad \frac{B_t}{B_T} = e^{-r(T-t)}.$$

La formule devient :

$$V_t = e^{-r(T-t)} \mathbb{E}^{\mathbb{Q}} [\Phi((S_u)_{t \leq u \leq T}) | \mathcal{F}_t].$$

Modélisation Monte Carlo

On estime le prix V_t en réalisant N simulations indépendantes de la trajectoire du sous-jacent :

$$(S_u^{(1)})_{t \leq u \leq T}, \dots, (S_u^{(N)})_{t \leq u \leq T}.$$

L'estimateur Monte Carlo du prix est :

$$\hat{V}_N = e^{-r(T-t)} \frac{1}{N} \sum_{i=1}^N \Phi \left((S_u^{(i)})_{t \leq u \leq T} \right).$$

Erreur standard de l'estimateur

On note les payoffs simulés :

$$Y_i = \Phi \left((S_u^{(i)})_{t \leq u \leq T} \right).$$

La moyenne empirique est :

$$\bar{Y}_N = \frac{1}{N} \sum_{i=1}^N Y_i.$$

La variance empirique est estimée par :

$$\hat{\sigma}^2 = \frac{1}{N-1} \sum_{i=1}^N (Y_i - \bar{Y}_N)^2.$$

$$\boxed{\widehat{\text{SE}} = e^{-r(T-t)} \frac{\hat{\sigma}}{\sqrt{N}}}.$$

Modèle de Black–Scholes et simulation

Sous la mesure risque-neutre $\mathbb{Q}$, le sous-jacent vérifie :

$$dS_t = rS_t dt + \sigma S_t dW_t^{\mathbb{Q}}.$$

Entre les dates t et T :

$$S_T = S_t \exp \left[\left(r - \frac{\sigma^2}{2} \right) (T - t) + \sigma \sqrt{T - t} Z \right], \quad Z \sim \mathcal{N}(0, 1).$$

Call européen

Pour un call européen, le payoff dépend seulement de S_T :

$$\Phi(S_T) = (S_T - K)^+ = \max(S_T - K, 0).$$

Son prix à l'instant t est :

$$V_t = e^{-r(T-t)} \mathbb{E}^{\mathbb{Q}} [(S_T - K)^+ \mid \mathcal{F}_t] .$$

Algorithme de Welford : mise à jour

L'algorithme de Welford calcule la moyenne et la variance empiriques en une seule passe, sans stocker tous les payoffs.

Initialisation :

$$n = 0, \quad \overline{Y}_0 = 0, \quad M_{2,0} = 0.$$

Pour chaque nouveau payoff Y_n , avec $n \geq 1$:

$$\delta_n = Y_n - \overline{Y}_{n-1},$$

$$\overline{Y}_n = \overline{Y}_{n-1} + \frac{\delta_n}{n},$$

$$M_{2,n} = M_{2,n-1} + \delta_n \left(Y_n - \overline{Y}_n \right).$$

Algorithme de Welford : variance et erreur standard

Après avoir traité les N payoffs simulés, l'algorithme fournit :

$$\overline{Y}_N = \frac{1}{N} \sum_{i=1}^N Y_i, \quad \hat{\sigma}_N^2 = \frac{M_{2,N}}{N-1}.$$

L'estimateur Monte Carlo du prix est alors :

$$\hat{V}_N = e^{-r(T-t)} \overline{Y}_N.$$

Son erreur standard est :

$$\widehat{\text{SE}} = e^{-r(T-t)} \frac{\hat{\sigma}_N}{\sqrt{N}}$$

Intervalle de confiance à 95 %

Pour un nombre de simulations N suffisamment grand, le théorème central limite donne approximativement :

$$\frac{\hat{V}_N - V_t}{\widehat{SE}} \sim \mathcal{N}(0, 1).$$

Comme une variable normale centrée réduite appartient à $[-1.96, 1.96]$ avec une probabilité proche de 95 % :

$$\text{IC}_{95\%} = \left[\hat{V}_N - 1.96 \widehat{SE}, \hat{V}_N + 1.96 \widehat{SE} \right].$$

Réduction de Variance : variables antithétiques

L'idée générale est de construire deux variables aléatoires ayant la même loi, mais négativement corrélées.

On note les deux payoffs :

$$X^+ \quad \text{et} \quad X^-.$$

Ils vérifient :

$$\mathbb{E}[X^+] = \mathbb{E}[X^-], \quad \text{Cov}(X^+, X^-) < 0.$$

Au lieu d'utiliser un seul payoff, on utilise leur moyenne :

$$X^{anti} = \frac{X^+ + X^-}{2}.$$

Réduction de variance : variables antithétiques

Dans le modèle de Black–Scholes :

$$S_T^+ = S_t \exp \left[\left(r - \frac{\sigma^2}{2} \right) (T - t) + \sigma \sqrt{T - t} Z \right],$$

$$S_T^- = S_t \exp \left[\left(r - \frac{\sigma^2}{2} \right) (T - t) - \sigma \sqrt{T - t} Z \right].$$

L'estimateur utilise ensuite la moyenne des deux payoffs :

$$\hat{V}_N^{anti} = e^{-r(T-t)} \frac{1}{N} \sum_{i=1}^N \frac{\Phi(S_{T,i}^+) + \Phi(S_{T,i}^-)}{2}.$$

Réduction de variance : variable de contrôle

L'idée est d'utiliser une variable aléatoire X dont l'espérance est connue, et qui est fortement corrélée avec le payoff Y .

On suppose connaître :

$$\mu_X = \mathbb{E}[X].$$

On construit alors une nouvelle variable :

$$Y^{ctrl} = Y - \beta (X - \mu_X).$$

Comme $\mathbb{E}[X - \mu_X] = 0$, l'espérance ne change pas :

$$\mathbb{E}[Y^{ctrl}] = \mathbb{E}[Y].$$

Pourquoi la variance peut diminuer ?

La variance de la variable corrigée est :

$$\text{Var}(Y^{ctrl}) = \text{Var}(Y) + \beta^2 \text{Var}(X) - 2\beta \text{Cov}(Y, X).$$

Si X est fortement corrélée avec Y , alors une partie des fluctuations de Y peut être expliquée par les fluctuations de X .

Le coefficient optimal est :

$$\beta^* = \frac{\text{Cov}(Y, X)}{\text{Var}(X)}.$$

Plus la corrélation entre X et Y est forte, plus la réduction de variance peut être importante.

Estimateur avec variable de contrôle

À partir de N simulations, on calcule :

$$Y_i^{ctrl} = Y_i - \beta (X_i - \mu_X).$$

L'estimateur Monte Carlo devient :

$$\widehat{V}_N^{ctrl} = \frac{1}{N} \sum_{i=1}^N Y_i^{ctrl}.$$

En pratique, β peut être estimé à partir des simulations :

$$\widehat{\beta} = \frac{\widehat{\text{Cov}}(Y, X)}{\widehat{\text{Var}}(X)}.$$

Application au call de Black–Scholes

Pour un call européen, le payoff actualisé :

$$Y_i = e^{-r(T-t)} \left(S_T^{(i)} - K \right)^+$$

Une variable de contrôle naturelle est le sous-jacent actualisé :

$$X_i = e^{-r(T-t)} S_T^{(i)}.$$

Sous la mesure risque-neutre, sans dividendes :

$$\mathbb{E}^{\mathbb{Q}}[X_i \mid \mathcal{F}_t] = S_t.$$

On utilise donc :

$$Y_i^{ctrl} = Y_i - \beta (X_i - S_t).$$

Pourquoi paralléliser Monte Carlo ?

Les simulations sont indépendantes :

$$Z_i \longrightarrow S_{T,i} \longrightarrow (Y_i, X_i), \quad i = 1, \dots, N.$$

Elles peuvent donc être réparties entre plusieurs cœurs du processeur.

Calculs locaux en parallèle	→	Fusion statistique
-----------------------------	---	--------------------

Répartition du travail avec OpenMP

OpenMP crée une équipe de threads et partage les simulations entre eux.

Thread	Simulations traitées
0	$1, \dots, N_0$
1	$N_0 + 1, \dots, N_1$
$\vdots$	$\vdots$
$p - 1$	$N_{p-2} + 1, \dots, N$

Chaque simulation est exécutée une seule fois.

Des données locales à chaque thread

Chaque thread possède :

- son propre générateur pseudo-aléatoire ;
- sa moyenne locale ;
- ses quantités locales de variance et de covariance.

$$\mathcal{S}_k = \left(n_k, \overline{Y}_k, \overline{X}_k, M_{2,Y,k}, M_{2,X,k}, C_{YX,k} \right).$$

Les threads travaillent ainsi sans modifier simultanément les mêmes données.

Fusion des statistiques de Welford

Pour fusionner deux groupes A et B , on pose :

$$n = n_A + n_B, \quad \delta_Y = \bar{Y}_B - \bar{Y}_A.$$

La moyenne et la dispersion globales sont :

$$\bar{Y} = \bar{Y}_A + \delta_Y \frac{n_B}{n}$$

$$M_{2,Y} = M_{2,Y,A} + M_{2,Y,B} + \delta_Y^2 \frac{n_A n_B}{n}$$

Le dernier terme mesure l'écart entre les moyennes des deux groupes.

Origine de la formule de fusion

Pour $i \in A$, on décompose :

$$Y_i - \bar{Y} = (Y_i - \bar{Y}_A) + (\bar{Y}_A - \bar{Y}).$$

Comme

$$\sum_{i \in A} (Y_i - \bar{Y}_A) = 0,$$

les termes croisés disparaissent. Ainsi :

$$M_{2,Y} = M_{2,Y,A} + M_{2,Y,B} + n_A(\bar{Y}_A - \bar{Y})^2 + n_B(\bar{Y}_B - \bar{Y})^2.$$

En remplaçant les deux écarts à la moyenne globale, on retrouve :

$$M_{2,Y} = M_{2,Y,A} + M_{2,Y,B} + \delta_Y^2 \frac{n_A n_B}{n}.$$

Fusion de la covariance

En posant $\delta_X = \bar{X}_B - \bar{X}_A$:

$$C_{YX} = C_{YX,A} + C_{YX,B} + \delta_Y \delta_X \frac{n_A n_B}{n}$$

Après la fusion de tous les threads :

$$\widehat{\text{Var}}(Y) = \frac{M_{2,Y}}{n-1}, \quad \widehat{\text{Cov}}(Y, X) = \frac{C_{YX}}{n-1}.$$

Les statistiques globales sont obtenues sans stocker les simulations.

Synchronisation des threads

`nowait`

Un thread ayant terminé ses simulations peut poursuivre sans attendre la fin des autres threads.

`critical`

La fusion avec l'état global est exécutée par un seul thread à la fois. Les autres threads attendent brièvement.

$\underbrace{\text{nombreuses simulations parallèles}}_N \longrightarrow \underbrace{\text{quelques fusions}}_p.$

Bilan de la parallélisation

- Les simulations sont réparties entre les threads.
- Chaque thread utilise un générateur et un état Welford privés.
- Les états locaux sont fusionnés mathématiquement.
- Une courte section critique protège la fusion globale.

L'accélération apportée par p threads se mesure par :

$$\text{Speedup}(p) = \frac{T_1}{T_p}$$

Elle reste généralement inférieure à p en raison des synchronisations et des limites matérielles.